

Liftoff X — Game Math, RTP & Volatility Sheet

Operator / aggregator technical reference. Unless marked [ASSUMPTION], every figure here is measured from the game's actual crash engine (`verify-fairness.mjs`) by Monte-Carlo simulation (N = 3,000,000 rounds) — not estimated. Player-behaviour / session items are marked [ASSUMPTION] and must be validated with live data. ⚠ Status disclaimer. Liftoff X uses a provably-fair RNG (HMAC-SHA256). It is not independently certified (e.g. GLI / iTech Labs) and holds no gaming licences. No certification or regulatory approval is claimed. Independent RNG certification is on the roadmap, not in place.

Game ID: `liftoffx` · Demo: <https://www.liftoffx.com> · Contact: stefanolijve85@gmail.com

At a glance

Attribute	Value
Category	Crash / instant multiplier game
Default RTP	≈ 97.0%
RTP options	97% / 96% / 95% (server-configurable)
House edge	≈ 3.0% / 4.0% / 5.0%
Volatility	High
Median crash	≈ 1.95×
Mean crash	≈ 21×
Max win	Uncapped by design (operator-configurable cap)
Min / max bet	Operator-configurable
Bets per round	Up to 2 simultaneous (dual bet) + auto-cashout
Round cadence	≈ 14.3 s / round (≈ 252 rounds / hour)
Fairness	Provably fair (HMAC-SHA256), player-verifiable

1. Game Overview

Liftoff X is a **crash game**: a rocket launches and a multiplier climbs from 1.00× upward; it crashes at a pre-committed, provably-fair point. Players cash out before the crash to win `stake × multiplier`; if the rocket crashes first, the stake is lost. Rounds are **server-authoritative and shared** — every connected player is in the same round (true multiplayer), with a live players count, activity feed, leaderboard and chat. Mobile-first web/PWA, multi-language, multi-currency-ready.

2. Core Gameplay Loop

- **Betting window — 5.0 s.** Player places up to **two** bets, each with an optional **auto-cashout** target.
- **Launch.** The multiplier rises exponentially: $M(t) = e^{(0.16 \cdot t)}$ (t in s).
- **Cash out** any time before the crash → win `stake × M`. Auto-cashout fires automatically at the chosen target.
- **Crash** at the round's pre-set crash point → any un-cashed bets lose.
- **Pause — 3.2 s** → next round begins. One shared clock for all players.

3. Crash Point Generation Logic

The crash point for round n is **fixed before the round starts**, from three inputs: a random **server Seed** (16 bytes; its SHA-256 hash is published in advance), a public **clientSeed**, and the **nonce** (round number).

```

digest = HMAC_SHA256(serverSeed, "clientSeed:nonce") // 256-bit hex

# Step 1 – instant-crash slice (the house edge)
hInt = first 32 bits of digest
if hInt mod K == 0: crash = 1.00× # default K = 50 → ~2% slice

# Step 2 – otherwise, the multiplier
h = first 52 bits of digest ; e = 2^52
crash = max(1.00, floor( (100·e - h) / (e - h) ) / 100)

```

Reference implementation: [verify-fairness.mjs](#) (identical in client & server). The Step-2 formula additionally floors to 1.00× across a small band of h , so the **effective** rate of 1.00× outcomes at the default config is $\approx 3.0\%$, not 2%.

4. RTP Configuration Options

RTP is tuned by a **single server-side parameter** (the instant-crash slice K), per operator / jurisdiction. Measured over 3,000,000 rounds each:

Config	Instant-slice	Effective 1.00× rate	Measured RTP	House edge
Default	$K = 50$ ($\sim 2\%$)	2.98%	$\approx 97.0\%$	$\approx 3.0\%$
Option B	$K \approx 33$ ($\sim 3\%$)	3.99%	$\approx 96.0\%$	$\approx 4.0\%$
Option C	$K = 25$ (4%)	4.95%	$\approx 95.0\%$	$\approx 5.0\%$

5. Default RTP

$\approx 97.0\%$. A defining property of this crash model: **RTP is $\sim$ constant across cashout strategies** (it does not reward or punish a particular target) — measured at the default config:

Cashout target	1.2×	1.5×	2.0×	3.0×	5.0×	10×	20×
RTP	97.85%	97.65%	97.45%	97.26%	97.20%	97.15%	96.99%

The slight drift is discretisation only. This uniformity is a transparency/fairness selling point: no "trap" cashout zones.

6. House Edge

$\approx 3.0\%$ at default ($= 1 - \text{RTP}$), implemented via the instant-crash slice plus the formula's 1.00× floor. Configurable to $\approx 4.0\%$ / 5.0% (RTP 96% / 95%). The house edge is structural and **independent of player skill or cashout choice**.

7. Volatility Rating

HIGH. Although the **median** crash is $\approx 1.95\times$, the **mean** is $\approx 21\times$: most rounds end low while a thin tail produces very large multipliers. Roughly **51% of rounds crash below 2.0×**, yet ~ 1 in 103 reaches $\geq 100\times$ and ~ 1 in 1,000 reaches $\geq 1,000\times$ (max observed in 3M rounds: $> 13,000,000\times$). This fat-tailed profile is the source of the game's excitement and its high variance.

8. Multiplier Distribution

$P(\text{crash} \geq m)$, measured at the default config (3M rounds):

m	1.0×	1.2×	1.5×	2×	3×	5×	10×	20×	50×	100×	1000×
$P(\geq m)$	100%	81.5%	65.1%	48.7%	32.4%	19.4%	9.71%	4.85%	1.93%	0.97%	0.10%

- Instant crash ($= 1.00\times$): **2.98%** · Median: $\approx 1.95\times$ · Mean: $\approx 21\times$.

9. Provably Fair Methodology

- **Commit:** the SHA-256 hash of [serverSeed](#) is shown **before** the round.
- **Play:** the crash point is HMAC-derived from [serverSeed](#) : [clientSeed](#) : [nonce](#).
- **Reveal:** after the round, [serverSeed](#) is revealed; anyone can recompute the HMAC and confirm the crash point — proving it was **not altered after bets**.
- Client, server, and the standalone [verify-fairness.mjs](#) verifier produce **byte-identical** results; an in-game "verify" view exposes every field.
- Provably fair is **complementary to** (not a substitute for) independent RNG certification; see the status disclaimer above.

10. Expected Return Calculations

For a cashout target m , the model gives $P(\text{reach } m) \approx \text{RTP} / m$, so the expected return $\text{EV} = P(\text{reach } m) \times m \approx \text{RTP}$ — **constant in m** . Worked example, default config, €1 stake:

Target	P(reach)	Payout if win	EV	Return
2.00×	48.7%	€2.00	€0.974	97.4%
5.00×	19.4%	€5.00	€0.972	97.2%
10.0×	9.71%	€10.00	€0.971	97.1%

Over volume, **operator GGR** → **turnover** × **house edge** regardless of how players play.

11. Example Simulations

Method: the production algorithm (`verify-fairness.mjs`), $N = 3,000,000$ rounds per config, fresh random serverSeed, public clientSeed `rocketrush-public-v1`.

Config	Instant 1.00×	Mean crash	$P(\geq 2\times)$	$P(\geq 10\times)$	$P(\geq 100\times)$	Avg RTP*
97% (default)	2.98%	21.2×	48.73%	9.71%	0.97%	97.4%
96%	3.99%	18.5×	48.27%	9.63%	0.96%	96.6%
95%	4.95%	13.9×	47.76%	9.51%	0.94%	95.4%

* mean of measured RTP across the 1.2×–20× cashout targets. Reproducible with the self-test (`node verify-fairness.mjs`).

12. Risk Profile

- **Per-round variance is high** (tail-driven). A single very-high multiplier is a large payout event.
- **For the operator:** by the law of large numbers, GGR **converges to $(1 - \text{RTP}) \times \text{turnover}$ as round volume grows; short-term swings come from the tail. With sufficient daily volume across a player base this is statistically stable.** Recommended control: **an operator-configurable max-win cap** to bound single-payout tail risk (this slightly raises effective RTP-to-operator and should be modelled per market).
- **For the player:** a high-variance, high-ceiling experience — the core appeal.

13. Session Metrics

Pacing is **measured** from the engine; engagement items are **[ASSUMPTION]** to be replaced with live operator data.

Metric	Value	Source
Full round cycle	14.3 s	measured
→ betting / flight / pause	5.0 s / 6.1 s / 3.2 s	measured
Rounds per hour (cadence)	≈ 252	measured
Rounds played / session	40–80	[ASSUMPTION]
Avg session length	15–30 min	[ASSUMPTION]
Avg stake / bet	€1.00	[ASSUMPTION] (see OPERATOR-ECONOMICS)

14. Average Round Duration

≈ **14.3 s** end-to-end: **5.0 s** betting + ≈ **6.1 s** mean flight + **3.2 s** pause. Flight time tracks the crash point (`flight = ln(crash) / 0.16 s`):

Flight time	p50	p75	p90	p99
seconds	4.2	8.5	14.2	28.8

(Median flight is short; the mean is pulled up by the tail, like the multiplier.)

15. Average Bets Per Hour

At ≈ 252 rounds/hour, a player who bets **every** round generates:

- **252 bets/hour** (single bet) — or **504 bets/hour** using both bet slots.

Real players bet a **fraction** of rounds, so effective bets/hour is lower (**[ASSUMPTION]**, validate live). The dual-bet design is a structural lever to lift bets-per-round above a single-bet crash game.

16. Revenue Model For Operators

GGR is driven directly by the math above: **GGR = turnover $\times$ house edge**.

Illustrative, **[ASSUMPTION]** €1 avg bet, RTP 96% (4% edge), one active player betting every round:

- Turnover/hour $\approx 252 \text{ rounds} \times €1 = \text{€}252 \rightarrow \text{GGR} \approx \text{€}10.1/\text{hour}$ to the operator.

Operator levers built into the product:

- **Configurable RTP** (97/96/95) $\rightarrow$ set the house edge per market.
- **Retention layer** (leaderboard, activity, social) $\rightarrow$ more rounds/player $\rightarrow$ more turnover $\rightarrow$ more GGR at the same hold.
- **Dual bets** $\rightarrow$ more bets per round.

The full commercial model — GGR per 1,000 players, **rev-share scenarios (8% / $\sim$ 11% / 15%)**, the tiered staffel, the "operator keeps more vs a 15% incumbent" comparison, and the portfolio roll-up — is in ``OPERATOR-ECONOMICS.md``. This sheet supplies the math that document's assumptions rest on; keep the two consistent.

Reproduce these figures

- Built-in self-test: `node verify-fairness.mjs` (distribution + house-edge sanity).
- The algorithm is identical in client and server; any third party can verify any round from `serverSeed`, `clientSeed`, `nonce`.

Document is a technical model of the current build; figures will be re-measured and an independent certification report substituted once certification is complete.